# Отчет по проекту **MirrorRoomSimulation**
# Симуляция отражений луча в зеркальной комнате

22 октября 2026 г.

## Содержание


**1 Общая характеристика проекта** **2**

**2 Возможности пользователя** **2**

2.1 Основной сценарий работы . . . . . 2

2.2 Что может делать пользователь . . . . . 3

2.3 Ограничения и обратная связь . . . . . 4

**3 Архитектура приложения** **4**

3.1 Пользовательский интерфейс . . . . . 5

3.2 Логика управления . . . . . 5

3.3 Геометрическая модель . . . . . 6

**4 Используемые технологии и темы** **6**

**5 Состав проекта** **8**

**6 Руководство пользователю** **8**

6.1 Построение комнаты . . . . . 8

6.2 Изменение зеркал . . . . . 8

6.3 Симуляция . . . . . 9

6.4 Сохранение истории . . . . . 9

**7 Руководство программисту** **9**

7.1 Сборка проекта . . . . . 9

7.2 Где менять параметры . . . . . 10

7.3 Как расширять проект . . . . . 10

7.4 На что обращать внимание . . . . . 11
7.5 Рекомендации по улучшению . . . . . 11

**8 Заключение** **11**

# 1 Общая характеристика проекта

Проект **MirrorRoomSimulation** представляет собой настольное приложение на C++ с использованием Qt 6 Widgets. Программа позволяет пользователю построить зеркальную комнату в виде многоугольника, задать типы стен, разместить источник луча и рассчитать траекторию отражений луча.

Основная идея приложения:

- пользователь строит комнату из точек и стен;
- стены могут быть плоскими, вогнутыми или выпуклыми зеркалами;
- после замыкания комнаты пользователь размещает источник луча на одной из стен;
- луч отражается от стен по закону отражения;
- программа показывает траекторию, анимирует ее и сохраняет историю отражений.

Проект можно рассматривать одновременно как:

- пользовательский интерактивный симулятор;
- учебный пример геометрических расчетов;
- пример организации кода на C++ и Qt;
- основу для дальнейшего развития оптического моделирования.

# 2 Возможности пользователя

## 2.1 Основной сценарий работы

Работа пользователя состоит из нескольких этапов: построение комнаты, замыкание контура, установка источника луча, задание угла, трассировка отражений и сохранение результата.

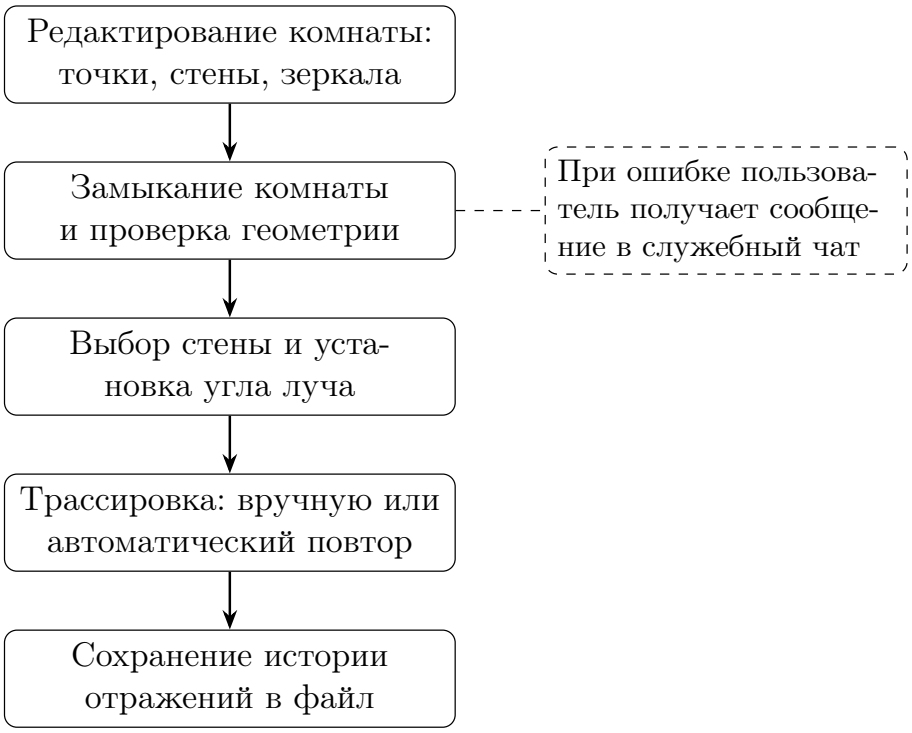



Рис. 1: Основной пользовательский сценарий

## 2.2 Что может делать пользователь

В режиме редактирования пользователь может:

- ставить новые точки кликом по рабочей области;
- изменять существующие точки;
- удалять точки;
- соединять текущую цепочку с предыдущей точкой;
- откатывать последнюю точку;
- менять тип зеркала и радиус кривизны стены.

После начала симуляции пользователь может:

- кликнуть по стене, чтобы разместить источник луча;
- задать угол луча относительно стены;
- выполнять отражения вручную;
- запускать автоматический повтор отражений;
- менять скорость анимации луча;
- менять задержку между отражениями;
- включать режим скрытия старых отражений;
- сохранять историю отражений в текстовый файл.

### 2.3 Ограничения и обратная связь

Программа ограничивает действия пользователя, чтобы геометрия оставалась корректной.

Ключевые ограничения:

- рабочая область имеет размеры от 0 до 1580 по $X$ и от 0 до 1000 по $Y$;
- количество сторон комнаты должно быть от 4 до 9;
- стены не должны пересекаться;
- стены не должны касаться друг друга вне общих вершин;
- точки не должны находиться слишком близко друг к другу;
- источник луча должен располагаться на стене;
- источник луча не может находиться слишком близко к вершине;
- угол луча должен быть от 0,01° до 179,99°;
- радиус криволинейного зеркала должен быть не меньше половины длины соответствующей стены.

Все ошибки и предупреждения выводятся в служебный чат справа. Это позволяет пользователю понимать, почему действие не было выполнено.

## 3 Архитектура приложения

Проект можно разделить на четыре основные части:

1. пользовательский интерфейс;
2. логика управления комнатой и трассировкой;
3. геометрическая модель;
4. константы, темы и оформление.




Рис. 2: Общая схема взаимодействия частей приложения

## 3.1 Пользовательский интерфейс

Интерфейс включает:

- главное окно с рабочей областью;
- служебный чат;
- кнопки управления режимами;
- ползунки скорости и задержки;
- поле ограничения числа отражений;
- модальные диалоги для ввода параметров.

Главное окно самостоятельно рисует сцену через `QPainter`: фон, рабочую зону, стены, точки, траекторию луча и источник.

## 3.2 Логика управления

Класс `FrontEndOperators` отвечает за:

- создание точек;
- удаление точек;
- изменение зеркал;

- замыкание комнаты;
- расчет следующего отражения;
- обнаружение зацикливания луча;
- хранение истории отражений.

Связь с интерфейсом выполнена через механизм уведомлений. Логика передает сообщение, а главное окно отображает его в чате.

### 3.3 Геометрическая модель

Геометрическая модель включает:

- класс точки `Points`;
- класс луча `Beam`;
- абстрактный класс зеркала `Mirror`;
- реализации плоского, вогнутого и выпуклого зеркал;
- класс многоугольника `Polygon`.

В этом слое выполняются расчеты пересечений, нормалей, отражений и проверки корректности комнаты.

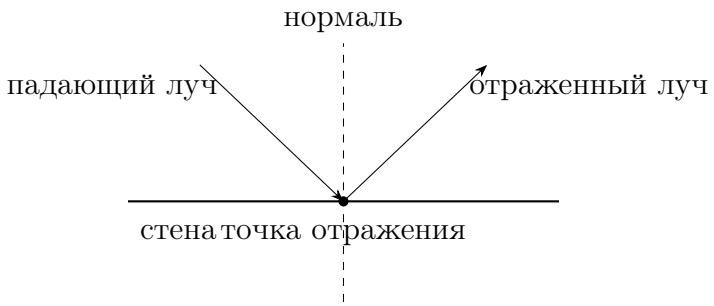



Рис. 3: Схема отражения луча от стены

## 4 Используемые технологии и темы

В проекте используются следующие технологии и темы программирования:

- C++17;
- Qt 6 Widgets;
- объектно-ориентированное программирование;
- наследование и полиморфизм для зеркал;

- умные указатели для управления стенами;
- сигналльно-слотовая модель Qt;
- пользовательская отрисовка через `QPainter`;
- анимация через `QVariantAnimation`;
- таймеры через `QTimer`;
- диалоговые окна и модальный ввод;
- файловый ввод-вывод через `QFile` и `QTextStream`;
- сборка через CMake;
- заготовка локализации через Qt Linguist.

Отдельно стоит отметить вычислительную геометрию:

- пересечение отрезков;
- пересечение отрезков и дуг;
- пересечение луча с отрезком;
- пересечение луча с дугой;
- вычисление нормалей;
- отражение направления луча;
- проверка многоугольника на самопересечения.

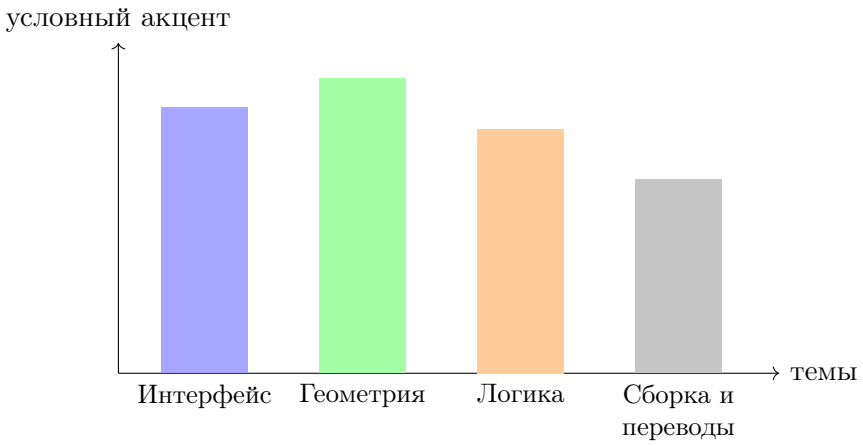



Рис. 4: Условная диаграмма акцентов проекта

# 5 Состав проекта

Ниже приведено краткое описание основных групп файлов.

| **Группа** | **Файлы и роль** |
|---|---|
| Точка входа | `main.cpp`: создание приложения, попытка загрузки перевода, запуск главного окна. |
| Интерфейс | `mainwindow.ui`, `mainwindow.h`, `mainwindow.cpp`; диалоги: `beamangledialog.*`, `changepointdialog.*`, `changeorconnectdialog.*`, `savehistorydialog.*`. |
| Логика управления | `frontendoperators.h`, `frontendoperators.cpp`: операции создания, удаления, изменения точек, замыкания и трассировки. |
| Геометрическая модель | `polygon.h`, `polygon.cpp`, `mirror.h`, `mirror.cpp`, `points.h`, `points.cpp`. |
| Константы и тема | `constants.h`: пределы, цвета, допуски, параметры анимации. `Darktheme.h`: стиль главного окна. |
| Сборка и переводы | `CMakeLists.txt`; файл перевода `MirrorRoomSimulation_ru_RU.ts`; `README.md`. |

# 6 Руководство пользователю

## 6.1 Построение комнаты

После запуска программа находится в режиме редактирования. Пользователь кликает по рабочей области, чтобы поставить точки. Между точками автоматически создаются плоские стены.

Клик по существующей точке открывает диалог выбора действия:

- изменить точку;
- подключить текущую цепочку к выбранной точке;
- удалить точку.

Доступные действия зависят от состояния комнаты. Например, в уже замкнутой комнате действие «Подключить» недоступно, так как контур уже замкнут.

## 6.2 Изменение зеркал

В диалоге изменения точки пользователь может задать:

- координату $X$;
- координату $Y$;

- тип зеркала;
- радиус кривизны для вогнутого или выпуклого зеркала.

Если радиус меньше половины длины стены, кривое зеркало невозможно. Программа оставит плоское зеркало и выведет предупреждение.

### 6.3 Симуляция

После нажатия кнопки начала симуляции программа замыкает комнату, если это возможно. Затем пользователь кликает по стене и задает угол луча.

Угол 90° означает, что луч направлен перпендикулярно стене. Для криволинейных стен используется касательная в точке источника.

После задания угла можно:

- выполнять отражения вручную;
- запустить автоматический повтор;
- остановить повтор;
- изменить угол луча кликом по красной точке источника.

### 6.4 Сохранение истории

Пользователь может сохранить историю отражений в текстовый файл. Перед сохранением программа спрашивает количество знаков после запятой для координат.

В файл записываются:

- дата и время сохранения;
- структура комнаты;
- параметры источника луча;
- точки отражений;
- углы направления луча после отражений.

## 7 Руководство программисту

### 7.1 Сборка проекта

Для сборки необходимы:

- компилятор с поддержкой C++17;
- CMake;

- Qt 6 с модулями Widgets и LinguistTools.

Типовая сборка:

```
cmake -S . -B build
cmake --build build
```

После сборки исполняемый файл будет находиться в каталоге сборки. Точное имя зависит от платформы.

## 7.2 Где менять параметры

Большинство настраиваемых значений находятся в файле `constants.h`. Там заданы:

- размеры рабочей области;
- минимальные расстояния;
- диапазон угла луча;
- скорость анимации;
- параметры чата;
- допуски геометрических расчетов;
- цвета темы.

Благодаря этому большинство числовых параметров не нужно искать по всему коду.

## 7.3 Как расширять проект

Для добавления новой функции программисту обычно нужно:

1. определить, к какому слою относится функция;
2. добавить данные в модель, если они нужны;
3. добавить операцию в логику;
4. добавить элемент интерфейса;
5. добавить сообщения для пользователя;
6. проверить ограничения и краевые случаи.

Например, добавление новой кнопки обычно включает:

- добавление кнопки в форму или код;
- добавление обработчика нажатия;
- обновление доступности кнопки;
- добавление сообщения в чат;
- проверку блокировок во время анимации.

## 7.4 На что обращать внимание

При сопровождении кода важно учитывать:

- геометрические расчеты зависят от числовых допусков;
- изменение стен требует проверки пересечений;
- изменение луча должно сбрасывать текущую трассировку;
- во время анимации многие действия должны блокироваться;
- история отражений может храниться даже после сброса визуального пути;
- пользовательские сообщения должны объяснять причину отказа.

## 7.5 Рекомендации по улучшению

Для дальнейшего развития проекта можно предложить:

- добавить сохранение и загрузку самой комнаты;
- добавить автоматические тесты для геометрии;
- уменьшить использование дружественных классов;
- перейти на более гибкие компоновки интерфейса;
- полноценно подготовить строки к локализации;
- добавить подсказки и горячие клавиши;
- ограничить рост истории при очень длинных траекториях;
- согласовать имя загружаемого перевода с фактическим файлом перевода.

# 8 Заключение

Проект **MirrorRoomSimulation** представляет собой законченное интерактивное приложение для моделирования отражений луча в зеркальной комнате. Он содержит пользовательский интерфейс, геометрическую модель, алгоритмы трассировки, анимацию, служебные сообщения и экспорт истории.

Для пользователя программа предоставляет понятный сценарий: построить комнату, задать зеркала, поставить источник, выполнить отражения и сохранить результат. Для программиста проект показывает пример организации Qt-приложения с разделением интерфейса, логики и геометрической модели.